

Parsing Compilers

Consider the following grammar:

$$E ::= E + E \mid E * E \mid (E) \mid num$$

1. Is this grammar ambiguous?
2. Executing the bottom-up parsing algorithm on the following expression: $0 * 1 * 2$
3. Explain where some ambiguity issue arises in your run of the algorithm

Solution

1. This grammar is ambiguous because $0 * 1 * 2$ could be parsed as $(0 * 1) * 2$ or $0 * (1 * 2)$.
2. Here is a Bottom-Up parsing of $0 * 1 * 2$:

step	stack	input	action
1	\$	$0 * 1 * 2$ \$	shift
2	$\$0$	$*1 * 2$ \$	reduce using $E ::= num$
3	$\$E$	$*1 * 2$ \$	shift
4	$\$E*$	$1 * 2$ \$	shift
5	$\$E * 1$	$*2$ \$	reduce using $E ::= num$
6	$\$E * E$	$*2$ \$	shift
7	$\$E * E*$	2 \$	shift
8	$\$E * E * 2$	\$	reduce using $E ::= num$
9	$\$E * E * E$	\$	reduce using $E ::= E * E$
10	$\$E * E$	\$	reduce using $E ::= E * E$
11	$\$E$	\$	accepted

3. The above run corresponds to the parsing of $0 * (1 * 2)$. If we had reduced instead of shifting in step 6, then we would have obtained a parsing of $(0 * 1) * 2$.